KARAKTERISTIK SET INSTRUKSI DAN ADDRESSING-MODE ORGANISASI DAN ARSITEKTUR KOMPUTER

Dosen Pengampu:
Yudi Sutanto, S.Kom., M.Kom.

Kelas: IF04
Kelompok 7

Ita Alfira Lala	(24.11.6127)
Adlia Syifa Zahara	(24.11.6146)
Ervila Nirmala	(24.11.6152)
Naura As-sheiha virdina Alyan	(24.11.6165)
Keisha Leana Allung	(24.11.6166)

S1-INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS AMIKOM YOGYAKARTA

2025/2026

BAB I
PENDAHULUAN

1.1 Latar Belakang

Instruksi merupakan komponen utama dalam sistem komputer yang berperan sebagai perintah dasar bagi CPU untuk menjalankan suatu proses. Seluruh aktivitas komputasi, mulai dari pemrosesan data sederhana hingga pengendalian alur program, dilaksanakan melalui eksekusi instruksi-instruksi mesin yang telah dirancang sesuai dengan arsitektur prosesor. Setiap instruksi tersusun atas beberapa elemen yang menentukan jenis operasi yang dilakukan serta cara data diakses dan diolah. Elemen-elemen tersebut mencakup kode operasi, operand, serta mekanisme pengalamatan yang digunakan. Perbedaan dalam struktur dan karakteristik instruksi akan mempengaruhi kinerja sistem, efisiensi penggunaan memori, serta kompatibilitas program pada suatu arsitektur komputer.

Oleh karena itu, pemahaman mengenai elemen-elemen instruksi, format instruksi, dan addressing mode menjadi hal yang penting dalam mempelajari arsitektur komputer. Dengan memahami konsep tersebut, mahasiswa diharapkan mampu menjelaskan cara kerja CPU dalam mengeksekusi instruksi serta memahami pertimbangan desain yang digunakan pada berbagai arsitektur prosesor modern.

1.2 Tujuan Tugas

Tujuan dari penyusunan tugas ini adalah sebagai berikut:

1. Memahami elemen-elemen dasar instruksi pada arsitektur komputer.
2. Menganalisis format instruksi serta variasi yang digunakan pada berbagai arsitektur.
3. Mengidentifikasi berbagai jenis addressing mode beserta cara penggunaannya.
4. Mengenali cakupan jenis instruksi seperti data transfer, arithmetic, logical dan control.
5. Menjelaskan contoh implementasi format instruksi dengan 3 alamat, 2 alamat, 1 alamat dan 0 alamat.

BAB II

ELEMEN–ELEMEN INSTRUKSI

2.1 Opcode (Operation Code)

Opcode atau Operation Code adalah bagian dari instruksi mesin yang berfungsi untuk menentukan jenis operasi yang harus dilakukan oleh CPU. Opcode menjadi penentu utama tindakan apa yang akan dieksekusi, sehingga CPU dapat mengatur kerja unit-unit internal seperti Arithmetic Logic Unit (ALU), register, dan unit kendali sesuai dengan perintah yang diberikan.

Fungsi utama opcode adalah sebagai pengenal operasi. Ketika sebuah instruksi diambil dari memori, CPU akan melakukan proses decoding terhadap opcode untuk mengetahui langkah-langkah eksekusi yang harus dijalankan. Tanpa opcode, CPU tidak dapat membedakan apakah suatu instruksi bertujuan untuk melakukan perhitungan, memindahkan data, atau mengendalikan alur program.

Jenis operasi yang bisa dipresentasikan:

a. Data Transfer: Memindahkan data antar lokasinya (LOAD, STORE, MOVE, PUSH dan POP).
b. Arithmetic: Operasi matematika, misalnya ADD, SUB, MUL, DIV, INC, DEC.
c. Logical: Operasi berbasis bit (AND, OR, NOT, XOR).
d. Control/Branch: mengubah urutan eksekusi program (JUMP, CALL, RET, Conditional Branches seperti JZ dan JNE).

2.2. Operand/Address Field

Operand atau address field merupakan bagian dari instruksi yang berfungsi untuk menunjukkan data yang akan diproses atau lokasi tempat data tersebut berada. Informasi ini sangat penting karena menentukan dari mana CPU mengambil data dan ke mana hasil operasi akan disimpan. Tanpa penentuan operand yang jelas, instruksi tidak dapat dieksekusi dengan benar oleh prosesor.

Lokasi Operand:

a. Register: penyimpanan internal CPU yang memiliki akses sangat cepat sehingga sering digunakan untuk operasi yang membutuhkan kecepatan tinggi.
b. Memori: memungkinkan pengolahan data dalam jumlah besar meskipun waktu aksesnya lebih lambat dibandingkan register.
c. Immediate Value: nilai data yang langsung dituliskan di dalam instruksi itu sendiri tanpa perlu mengambilnya dari register atau memori.

Hubungan antara operand dengan addressing mode sangat kuat, karena addressing mode menentukan bagaimana lokasi operand tersebut ditentukan dan diakses oleh CPU. Setiap addressing mode memberikan cara yang berbeda dalam menafsirkan address field, misalnya apakah alamat yang tertulis menunjuk langsung ke data, ke register tertentu, atau masih memerlukan perhitungan tambahan. Dengan demikian, pemilihan addressing mode berpengaruh langsung terhadap fleksibilitas pemrograman, efisiensi penggunaan memori, dan kecepatan eksekusi instruksi.

2.3 Mode Field (Addressing Mode Field)

Mode field berfungsi untuk menentukan bagaimana CPU menafsirkan operand yang tercantum dalam instruksi. Addressing mode menjelaskan cara memperoleh nilai operand yang sebenarnya, apakah langsung digunakan, diambil dari register, atau diakses melalui alamat memori tertentu. Misalnya, jika Operand Field berisi nilai 1000:

- Jika Mode = Immediate, operand adalah nilai 1000.
- Jika Mode = Direct, operand berada di alamat memori 1000.
- Jika Mode = Indirect, operand berada di alamat memori yang tersimpan di alamat 1000 (Mem[Mem[1000]]).

2.4 Register Field

Register field merupakan bagian dari instruksi yang digunakan untuk menunjukkan register mana yang terlibat dalam suatu operasi. Melalui register field, CPU dapat mengetahui lokasi penyimpanan operand maupun tempat penyimpanan hasil operasi yang akan dilakukan. Register yang ditunjuk dapat berupa register sumber, register tujuan, atau keduanya, tergantung pada format instruksi yang digunakan. Dengan mencantumkan register secara eksplisit di dalam instruksi, CPU dapat melakukan akses data dengan cepat karena register berada di dalam prosesor dan memiliki waktu akses yang jauh lebih singkat dibandingkan memori utama. Dalam arsitektur RISC modern (seperti MIPS, RISC-V), sebagian besar instruksi memiliki beberapa Register Field karena operasinya umumnya berbasis register.

2.5 Flag/Kondisi

Flag atau kondisi merupakan bagian dari status register yang mencerminkan hasil suatu operasi. Beberapa flag yang umum digunakan antara lain Zero (Z), Carry (C), dan Overflow (V). Flag adalah bit dalam Status Register (juga dikenal sebagai Program Status Word atau PSW) yang disetel atau dihapus oleh CPU setelah eksekusi instruksi. Flag ini mencerminkan kondisi hasil operasi terakhir:

a. Zero (Z): Disetel (1) jika hasil operasi adalah nol.
b. Carry (C): Disetel (1) jika terjadi carry out dari bit paling signifikan (digunakan dalam aritmetika presisi ganda/multi-word).

c. Overflow (V): Disetel (1) jika terjadi overflow dalam aritmatika bilangan bertanda.
d. Sign/Negative (N): Disetel (1) jika hasil operasi negatif.

2.6 Contoh Representasi dalan ISA Nyata

Dalam implementasinya, elemen-elemen instruksi dapat dilihat secara nyata pada berbagai arsitektur komputer. Perbedaan representasi ini menunjukkan bagaimana elemen-elemen instruksi disesuaikan dengan filosofi desain masing-masing arsitektur, baik yang menekankan kesederhanaan maupun fleksibilitas.

Fitur	MIPS	RISC-V	x86
Panjang Instruksi	Fixed (32-bit)	Fixed (32-bit, 64-bit)	Variable (1-15 bytes).
Opcode	Ukuran tetap, 6 bit pertama.	Ukuran tetap, 7 bit pertama	Ukuran bervariasi (1-2 byte) dan sering dikombinasikan dengan field lain
Operand Field	Berbasis register.	Berbasis register.	Kombinasi register, memori dan immediate. Dapat memiliki 0,1 atau 2 operand.
Addressing Mode	Sangat terbatas	Sangat terbatas.	Sangat kompleks dan bervariasi.
Flag	Tidak ada set/penggunaan flag eksplisit untuk percabangan. Percabangan didasarkan pada perbandingan dua register (BEQ, BNE).	Tidak ada set/penggunaan flag eksplisit untuk percabangan.	Menggunakan Flag (Zero, Carry, Overflow, Sign, Parity) di register EFLAGS. Percabangan bersyarat (JCC) didasarkan pada status flag.

BAB III
REPRESENTASI INSTRUKSI

3.1 Representasi Biner

Representasi biner adalah cara paling mendasar sebuah komputer berpikir, menyimpan data, dan memproses informasi. Jika user menggunakan sistem bilangan desimal (basis 10, angka 0-9), komputer menggunakan sistem biner (basis 2, hanya angka 0 dan 1).

Contoh:

a. Struktur bit (misalnya 6-bit opcode + 5-bit register, dll).
    1) Opcode (6 bit) menentukan operasi yang dilakukan CPU dengan 6 bit yang dapat merepresentasikan 2^6 = 64 instruksi yang berbeda.
    2) Register (5 bit) menunjukkan register tujuan atau sumber dengan 5 bit yang dapat merepresentasikan 2^5 = 32 register.
    3) Operand/Address (5 bit) yang bisa berisi alamat memori, data langsung (immediate), dan register tambahan.

3.2 Representasi Hexadecimal

Hexadecimal adalah representasi bilangan basis 16 yang digunakan sebagai bentuk singkat dari biner, dimana setiap digit hex mewakili 4 bit biner. Sistem ini digunakan dalam membaca alamat memori, instruksi mesin, dan data tingkat rendah. Keuntungan dari representasi hexadecimal yaitu menjadi lebih ringkas daripada ke biner, mudah dibaca, dan banyak dipakai di debugging, embedded system, dan arsitektur komputer.

a. Hubungan hexadesimal dengan biner
    Contoh:
    Biner : l101 1010 0111
    Hex : D A 7
b. Contoh konversi biner ke hexa
    Biner : 1010 1111
    Hex : A F
c. Contoh konversi hexa ke biner
    Hex : 3C
    Biner : 0011 1100
d. Contoh konversi desimal ke hexa
    Decimal : 26
    Hex : 1A

3.3 Representasi Bahasa Mesin (Machine Instruction)

Bahasa mesin (machine instruction) adalah bahasa tingkat paling rendah yang dapat langsung dipahami dan dieksekusi oleh CPU. Instruksi ini direpresentasikan dalam bentuk kode biner (0 dan 1) atau sering dituliskan dalam heksadesimal agar lebih mudah dibaca.

Komponen utama instruksi mesin terdiri dari beberapa field bit, yaitu:

a. **Opcode**
   Opcode adalah kode numerik atau biner yang merupakan bagian dari instruksi bahasa mesin yang memberi tahu unit pemrosesan pusat (CPU) operasi spesifik apa yang harus dilakukan, seperti penjumlahan, pemindahan data, atau logika lainnya.
b. **Operand**
   Operand adalah data, nilai, variabel, atau objek yang menjadi subjek operasi atau tindakan yang dilakukan oleh sebuah operator dalam komputasi atau matematika.
c. **Addressing Mode**
   Addressing mode adalah metode yang digunakan CPU untuk menentukan lokasi data (operand) yang dibutuhkan untuk suatu instruksi, dengan cara mengimpresentasikan atau memodifikasi bagian alamat dalam instruksi tersebut sebelum mengakses operan di memori atau register.

3.4 Representasi Assembly (Symbolic Instruction)

Bahasa Assembly (symbolic instruction) adalah representasi simbolik dari bahasa mesin. Instruksi ini berupa kode biner dituliskan dalam bentuk simbol agar lebih mudah dipahami. Assembly menjadi jembatan antara bahasa mesin dan bahasa tingkat tinggi. Struktur instruksi assembly terdiri dari LABEL (opsional, nama untuk alamat), kode operasi (Opcode), (mnemonik instruksi seperti MOV, ADD), Operand(data atau register yang dioperasikan, dipisahkan koma), dan komentar (teks deskriptif).

Contoh instruksi assembly umum:

Assembly	Fungsi
MOV R1, R2	Salin data R2 ke R1
ADD R3, R1, R2	R3 = R1 + R2
SUB R1, R1, #1	R1 = R1 - 1
JMP LOOP	Lompat ke label LOOP

BAB IV
FORMAT INSTRUKSI

4.1 Instruksi Panjang Tetap (Fixed-Length Instruction Format)

Instruksi panjang tetap (fixed-length instruction format) adalah format instruksi dimana setiap instruksi memiliki jumlah bit yang sama, tanpa tergantung jenis operasinya. Instruksi ini digunakan pada arsitektur RISC karena instruksi yang sederhana membuat dekoding menjadi lebih mudah dan waktu eksekusi lebih dapat diprediksi.

1. Contoh pada MIPS
   Panjang instruksi : 32 bit untuk semua instruksi
   a. R-Type (Register)
      Add $t0, $t1, $t2 > $t0 = $t1 + $t2
   b. Format Instruksi MIPS
      Opcode (6)
      Rs (5)
      Rt (5)
      Rd (5)
      Shamt (5)
      Funct (6)
   c. Nilai setiap field

Field	Nilai	Keterangan
opcode	000000	R-Type
rs	01001	$t1
rt	01010	$t2
rd	01000	$t0
shamt	00000	Tidak shift
funct	100000	ADD

2. Contoh pada RISC-V
   a. Instruksi Assembly
      Add x5, x6, x7 > x5 = x6 + x7
   b. Format Instruksi RISC-V
      Funct7 (7)
      Rs2 (5)
      Rs1 (5)

Funct3 (3)
Rd (5)
Opcode (7)

c. Nilai Field

Field	Nilai
funct7	0000000
rs2	00111 (x7)
rs1	00110 (x6)
funct3	000
rd	00101 (x5)
opcode	0110011

4.2 Instruksi Panjang Variabel (Variable-Length Instruction Format)

Instruksi panjang variabel (variable-length instruction format) adalah format instruksi di mana panjang instruksi tidak tetap, dapat berbeda-beda tergantung jenis operasi dan operand yang digunakan. Dalam satu arsitektur, instruksi bisa berukuran pendek atau panjang, misalnya 1 hingga belasan byte.

Contoh arsitektur: x86

Arsitektur x86 (CISC) menggunakan instruksi panjang variabel. Panjang instruksi x86 yaitu 1-15 byte dan satu instruksi bisa melakukan operasi kompleks sekaligus. Instruksi x86 terdiri dari beberapa bagian opsional yaitu:

1. Prefix
2. Opcode
3. ModR/M
4. SIB
5. Displacement
6. Immediate

4.3 Bagian-bagian Format

a. **Opcode Field**

Opcode adalah bagian instruksi yang menggunakan operasi apa yang akan dilakukan oleh CPU. Fungsi opcode yaitu untuk menentukan jenis instruksi dan mengendalikan unit kontrol CPU. Contoh: ADD, R1, R2

b. **Mode Field**

Mode field menentukan cara CPU dalam mengakses operand. Fungsi mode field yaitu untuk menjelaskan apakah operand berupa register, alamat memori, atau nilai langsung (immediate), dan menghemat bit instruksi dengan fleksibilitas akses data.

c. **Register Field**

Register Field adalah register CPU yang digunakan sebagai sumber atau tujuan data. Fungsi register field yaitu untuk menentukan lokasi data di dalam CPU dan mempercepat akses data dibanding memori.

Contoh: ADD, R3, R1, R2

R2 : register tujuan

R1, R2 : register sumber

d. **Memory Address Field**

Memory address field berisi alamat lokasi data atau instruksi di memori utama, Fungsi memory address field yaitu menentukan lokasi data yang akan dibaca atau ditulis dan digunakan pada instruksi LOAD, STORE, JUMP.

Contoh: LOAD, R1, 1024

1024 : alamat memori

e. **Displacement/Offset**

Displacement/offset adalah nilai tambahan yang digabungkan dengan register atau alamat dasar untuk menentukan alamat efektif. Fungsi displacement/offset yaitu untuk mendukung pengalaman relatif.

Contoh: LOAD R1, 8(R2)

Alamat efektif : isi R2 + 8

BEQ R1, R2, +16

+16 : offset relatif terhadap PC

f. **Immediate Value**

Immediate value adalah data langsung yang disimpan di dalam instruksi, bukan di memori atau register. Fungsi immediate value yaitu mempercepat operasi sederhana dan mengurangi akses memori.

Contoh: ADDI R1, R2, #10

10 : immediate value

Contoh biner: 00001010

4.4 Diagram Format Instruksi (Contoh MIPS/RISC-V)

a. **R-format**

opcode	rs	rt	rd	shamt	funct
6 bit	5 bit	5 bit	5 bit	5 bit	6 bit

b. I-format

opcode	rs	rt	immediate
6 bit	5 bit	5 bit	16 bit

c. J-format

opcode	address
6 bit	26 bit

BAB V
BENTUK SIMBOLIK INSTRUKSI

Dalam arsitektur komputer, komunikasi antara perangkat lunak dan perangkat keras dijembatani oleh set instruksi (instruction set). Karena bahasa mesin (machine code) yang berbasis biner sangat kompleks untuk dipahami manusia secara langsung, dikembangkanlah bentuk simbolik yang disebut Bahasa Assembly. Bab ini akan membahas secara rinci mengenai notasi simbolik instruksi, metode pengalamatan operand, serta mekanisme konversi dari bentuk simbolik ke bentuk biner yang dapat dieksekusi oleh *Central Processing Unit* (CPU).

5.1 Notasi Assembly

Notasi assembly merupakan abstraksi tingkat rendah (*low-level abstraction*) dari kode mesin. Setiap pernyataan dalam bahasa assembly umumnya berkorespondensi satu-satu dengan instruksi mesin. Struktur dasar instruksi assembly terdiri dari dua komponen utama: Mnemonic dan Operand.

5.1.1 Mnemonic (Opcode)

Mnemonic adalah representasi tekstual dari kode operasi (*Operation Code* atau *Opcode*). Secara teoritis, mnemonic berfungsi sebagai perintah simbolik yang memberitahu *Control Unit* pada CPU mengenai jenis operasi aritmatika atau logika yang harus dijalankan. Penggunaan mnemonic didasarkan pada singkatan kata kerja dalam bahasa Inggris untuk memudahkan *human-readability*. Contoh standar meliputi:

- **Transfer Data: MOV** (Move), **PUSH**, **POP**.
- **Aritmatika: ADD** (Addition), **SUB** (Subtraction), **INC** (Increment).
- **Kontrol Aliran: JMP** (Jump), **CMP** (Compare), **JE** (Jump if Equal).

5.1.2 Operand

Operand adalah entitas data yang menjadi objek dari operasi yang didefinisikan oleh mnemonic. Dalam satu instruksi, jumlah operand dapat bervariasi (nol, satu, dua, atau tiga) tergantung pada arsitektur prosesor (misalnya x86 atau MIPS). Dalam instruksi dua operand (Format: Mnemonic Destination, Source), operand dibedakan menjadi:

1. Source Operand (Asal): Sumber data yang akan diproses. Nilainya tidak berubah setelah operasi (kecuali jika source sama dengan destination).
2. Destination Operand (Tujuan): Lokasi penyimpanan hasil operasi.

5.2 Klasifikasi Penulisan Operand (Addressing Modes)

Efisiensi eksekusi instruksi sangat bergantung pada bagaimana CPU mengakses data. Mekanisme penentuan lokasi data ini disebut sebagai Mode Pengalamatan (Addressing Modes). Berikut adalah tiga mode fundamental:

5.2.1 Register-based Addressing (Pengalamatan Register)

Pada mode ini, operand merujuk langsung pada register internal CPU (seperti AX, BX, R1, dsb). Ini adalah metode pengalamatan tercepat. Karena data sudah berada di dalam prosesor, CPU tidak perlu mengakses system bus untuk mengambil data dari memori eksternal, sehingga meminimalisir latency.

Contoh Instruksi: *ADD AX, BX*

=> *Tambahkan isi register BX ke AX, dan simpan hasilnya di AX.*

5.2.2 Immediate-based Addressing (Pengalamatan Segera)

Mode ini digunakan ketika nilai data (konstanta) disisipkan secara langsung di dalam instruksi itu sendiri saat proses instruction fetch. Nilai operand dianggap sebagai bagian dari instruksi, bukan sebagai alamat. Mode ini efisien untuk inisialisasi variabel atau operasi matematika dengan konstanta tetap. Kelemahannya adalah nilai data bersifat statis (hardcoded) dan rentan membatasi fleksibilitas kode.

Contoh Instruksi: *MOV CX, 10*

=>*Masukkan nilai desimal 10 secara langsung ke register CX.*

5.2.3 Memory-based Addressing (Pengalamatan Memori/Langsung)

Pada mode ini, operand menunjuk pada alamat lokasi di memori utama (RAM).CPU harus menghitung Effective Address (EA) untuk menemukan lokasi data. Mode ini lebih lambat dibandingkan mode register karena melibatkan siklus pembacaan memori (memory read cycle) melalui bus data. Dalam notasi assembly, ini sering ditandai dengan penggunaan kurung siku [...].

Contoh Instruksi: *MOV AX, [2050]*

=> *Salin data yang berada di alamat memori 2050 ke dalam register AX.*

5.3 Konversi Assembly ke Biner

Proses penerjemahan dari notasi simbolik (Assembly) menjadi kode mesin (Biner) dilakukan oleh program utilitas yang disebut **Assembler**. Proses ini krusial karena CPU hanya dapat memproses sinyal listrik yang direpresentasikan oleh bit 1 (tegangan tinggi) dan 0 (tegangan rendah).

5.3.1 Mekanisme Translasi

Setiap instruksi assembly diterjemahkan menjadi urutan bit yang terbagi dalam beberapa *field* instruksi:

1. **Field Opcode:** Mnemonic diterjemahkan menjadi kode biner unik operasi.
2. **Field Register/Mode:** Menentukan register mana yang digunakan atau mode pengalamatan apa yang dipakai.
3. **Field Operand/Displacement:** Menyimpan nilai immediate atau alamat memori.

5.4 Studi Kasus Konversi

Sebagai ilustrasi teoritis pada arsitektur 16-bit hipotetikal, tinjau instruksi: MOV AL, 5 Proses konversi bertahap:

1. Analisis Mnemonic: Instruksi MOV (move to register 8-bit) memiliki Opcode biner, misalnya: 10110.
2. Analisis Destination: Register AL memiliki identitas biner: 000.
3. Analisis Source: Nilai 5 desimal dikonversi ke biner 8-bit: 00000101.

Komponen Instruksi	Notasi Simbolik	Representasi Biner (Machine Code)
Opcode	MOV (Immediate)	10110
Register	AL	0
Immediate Data	5	101

BAB VI
CAKUPAN JENIS INSTRUKSI

Set Instruksi (Instruction Set) merupakan kumpulan perintah dasar yang dapat dimengerti dan dieksekusi oleh sebuah prosesor. Meskipun arsitektur komputer (seperti x86, ARM, atau MIPS) memiliki sintaks assembly yang berbeda, secara teoritis instruksi-instruksi tersebut dapat diklasifikasikan ke dalam beberapa kategori fungsional utama. Bab ini akan membahas secara komprehensif mengenai enam kategori utama instruksi: perpindahan data, aritmatika, logika, kontrol aliran, kontrol prosesor, dan input/output.

6.1 Data Movement Instruction (Instruksi Perpindahan Data)

Kelompok instruksi ini bertanggung jawab untuk menyalin data dari satu lokasi ke lokasi lain (register-ke-register, memori-ke-register, atau sebaliknya) tanpa mengubah nilai data tersebut.

1. MOV (Move): Merupakan instruksi paling fundamental. Secara teoritis, MOV menyalin data dari operand sumber (source) ke operand tujuan (destination).
   Contoh: MOV AX, BX (Salin isi BX ke AX).
2. LOAD (Load Data): Umumnya ditemukan pada arsitektur Load-Store (seperti RISC/MIPS). Instruksi ini secara spesifik mengambil data dari memori utama (RAM) dan menempatkannya ke dalam register CPU.
   Fungsi: Menyiapkan data untuk diproses oleh ALU.
3. STORE (Store Data): Kebalikan dari LOAD. Instruksi ini memindahkan hasil pemrosesan dari register kembali ke alamat tertentu di memori utama untuk disimpan.
4. PUSH (Push to Stack): Menyimpan data ke dalam struktur data tumpukan (Stack). Secara teknis, instruksi ini akan mengurangi nilai Stack Pointer (SP) dan menempatkan data di alamat yang ditunjuk oleh SP.
5. POP (Pop from Stack): Mengambil data dari puncak tumpukan (Top of Stack) dan memindahkannya ke register. Instruksi ini akan menambah nilai Stack Pointer setelah data diambil.

6.2 Arithmetic Instruction (Instruksi Aritmatika)

Instruksi ini melibatkan *Arithmetic Logic Unit* (ALU) untuk melakukan operasi matematis dasar. Hasil dari operasi ini sering kali mempengaruhi *Flags Register* (seperti Zero Flag, Carry Flag, atau Overflow Flag).

1. ADD (Addition): Menjumlahkan dua operand. Hasil penjumlahan disimpan di operand tujuan.
   Logika: $Dest \leftarrow Dest + Source$
2. SUB (Subtraction): Mengurangkan operand sumber dari operand tujuan. Sering digunakan juga untuk membandingkan dua nilai.
   Logika: $Dest \leftarrow Dest - Source$

3. MUL (Multiplication): Mengalikan dua nilai. Pada banyak arsitektur, hasil perkalian mungkin membutuhkan ruang bit dua kali lebih besar dari operand (misal: 16-bit x 16-bit = 32-bit result), sehingga sering melibatkan pasangan register.
4. DIV (Division): Membagi nilai pada register tertentu dengan operand. Hasilnya biasanya dipecah menjadi dua bagian: hasil bagi (quotient) dan sisa bagi (remainder) yang disimpan di register terpisah.

6.3 Logical Instruction (Instruksi Logika)

Instruksi ini melakukan operasi *bitwise* (manipulasi level bit) berdasarkan aljabar Boolean. Instruksi ini krusial untuk manipulasi masker data (*masking*) dan pengujian bit.

1. **AND**: Melakukan operasi logika AND bit-demi-bit. Menghasilkan 1 hanya jika kedua bit operand bernilai 1. Sering digunakan untuk *masking* (memaksa bit tertentu menjadi 0).
2. **OR**: Melakukan operasi logika OR. Menghasilkan 1 jika salah satu atau kedua bit operand bernilai 1. Digunakan untuk men-set bit tertentu menjadi 1.
3. **NOT**: Melakukan inversi atau negasi (Komplemen 1). Mengubah bit 1 menjadi 0 dan sebaliknya.
4. **XOR** (Exclusive OR):Menghasilkan 1 jika kedua bit operand berbeda. Jika kedua bit sama, hasilnya 0. Cara efisien untuk mengosongkan register (XOR register dengan dirinya sendiri menghasilkan 0) atau mengenkripsi data sederhana.

6.4 Control Flow Instruction (Instruksi Kontrol Aliran)

Secara default, CPU mengeksekusi instruksi secara berurutan (*sequential*). Instruksi kontrol aliran berfungsi untuk memutus urutan tersebut dengan mengubah nilai pada *Program Counter* (PC), memungkinkan terjadinya percabangan (*branching*), pengulangan (*looping*), dan pemanggilan fungsi.

1. JMP (*Jump*): Lompatan tanpa syarat (*Unconditional Branch*). CPU dipaksa pindah ke alamat instruksi target seketika.
2. BEQ (*Branch if Equal*): Lompatan bersyarat. CPU akan lompat hanya jika hasil perbandingan sebelumnya (biasanya instruksi CMP) menunjukkan nilai "Sama Dengan" ($Zero Flag = 1$).
3. BNE (*Branch if Not Equal*): Kebalikan dari BEQ. CPU lompat jika nilai perbandingan "Tidak Sama".
4. CALL: Digunakan untuk memanggil sub-rutin (prosedur/fungsi). Sebelum lompat, CPU akan menyimpan alamat instruksi berikutnya ke dalam Stack agar bisa kembali nanti.
5. RET (*Return*): Digunakan di akhir sub-rutin untuk kembali ke program utama. CPU mengambil alamat yang disimpan oleh CALL dari Stack dan memasukkannya kembali ke *Program Counter*.

6.5 Processor Control Instruction (Instruksi Kontrol Prosesor)

Instruksi ini tidak memproses data aplikasi, melainkan mengatur status dan kondisi kerja CPU itu sendiri.

1. SETFLAG (STC/STD dll): Mengubah status bit tertentu pada Flag Register menjadi 1. Contoh: STC (Set Carry Flag).

2. CLEARFLAG (CLC/CLD dll): Mengubah status bit tertentu pada Flag Register menjadi 0. Contoh: CLC (Clear Carry Flag).

3. NOP (No Operation): Instruksi yang memerintahkan CPU untuk tidak melakukan apa-apa selama satu siklus instruksi. Sering digunakan untuk *padding* (pengisi) memori agar alamat instruksi sejajar (*alignment*) atau untuk membuat penundaan waktu (*delay*) yang sangat singkat.

6.6 I/O Instruction (Instruksi Input/Output)

Pada arsitektur yang menggunakan *Isolated I/O* (seperti x86), instruksi khusus diperlukan untuk berkomunikasi dengan perangkat periferal melalui port I/O.

1. **IN (Input):** Membaca data dari port perangkat keras tertentu dan memuatnya ke dalam register CPU (biasanya Akumulator).

   Contoh: Membaca penekanan tombol dari *keyboard buffer*.

2. **OUT (Output):** Mengirimkan data dari register CPU ke port perangkat keras tertentu.

   Contoh: Mengirim data karakter untuk ditampilkan ke layar monitor atau dikirim ke printer.

BAB VII
INSTRUCTION ADDRESSING MODE

1. Immediate Addressing

Data ini adalah bagian dari instruksi itu sendoro. Tidak ada pencarian ke memori atau register.

Cara kerjanya : Nilai langsung dikodekan ke dalam instruksi saat kompilasi

Kelebihan : Ukuran data terbatas dan nilainya statis (tidak bisa diubah saat program berjalan).

2. Register Addressing

Data disimpan di dalam register internal CPU (AX, BX, CX, DX)

Cara Kerja : CPU hanya perlu mengakses sirkuit internalnya sendiri.

Kelebihan : Paling cepat setelah Immediate, karena tidak melibatkan bus data eksternal.

3. Direct Addressing

Alamat memori data terkandung secara eksplisit dalam instruksi

Cara Kerja : CPU melihat instruksi, mengambil alamatnya, lalu melakukan satu kali akses ke RAM untuk mengambil data di alamat tersebut.

4. Indirect Addressing

Instruksi menentukan register atau lokasi memori yang berisi alamat dari data yang sebenarnya.

Cara Kerja : Seperti mencari harta karun dengan peta, instruksi memberi lokalisasi alamat dan di lokasi tersebut ada alamat asli datanya.

Kelebihan : Sangat fleksibel untuk pemrograman dinamis.

5. Displacement / Indexed Addressing

Alamat efektif ditemukan dengan menambahkan nilai konstanta isi register indeks.

Cara Kerja : *Effective Address = Register + Displacement*

Kegunaan : Sangat penting untuk mengakses array. Jikan SI adalah awal array, displacement adalah indeks elemennya.

6. Base-Register Addressing

Mirip dengan Indexed, tetapi menggunakan register basis (BX atau BP)

Cara Kerja: Sering digunakan untuk mengakses data dalam stack frame atau struktur data.

Rumus: *Alamat = Base Register + Offset*

7. **Relative Addressing**
   Alamat ditentukan relatif terhadap posisi instruksi yang sedang dieksekusi (Program Counter/Instruction Pointer)
   Cara Kerja: Alamat sasaran = *IP* + *Offest*
   Kegunaan: Digunakan untuk instruksi percabangan (JUMP, LOOP, CALL). Hal ini membuat kode bersifat "position Independent" (bisa berpindah-pindah di memori tanpa merusak logika lompatan)

8. **Stack Addressing**
   Mode pengalamatan yang beroperasi pada struktur data stack (LIFO menggunakan register khusus SP (Stack Pointer).
   Cara Kerja: Alamat tidak disebutkan secara otomatis merujuk pada puncak stack

9. **Contoh per Addressing Mode dalam kode assembly**

Mode	Addressing Code	Lokasi
Immediate	**MOV AX, 10**	Di dalam instruksi
Register	**MOV AX, BX**	Di dalam Register
Direct	**MOV AX, [2000h]**	Di alamat memori spesifik
Indirect	**MOV AX, [BX]**	Di alamat yang disimpan di BX
Indexed	**MOV AX, [SI + 5]**	Register Index konstanta
Stack	**PUSH AX**	Di puncak memori Stack

BAB VIII
FORMAT INSTRUKSI 3 ALAMAT, 2 ALAMAT, 1 ALAMAT, 0 ALAMAT

1. **Format Instruksi 3 Alamat**
   Format ini adalah paling lengkap karena dalam satu instruksi terdapat alamat dua sumber operan dan satu alamat untuk hasil akhirnya.
   Contoh: *ADD R1, A, B*
   (Ambil data dari lokasi A, ditambahkan dengan data di lokasi B, lalu simpan hasilnya di register R1)
   Kelebihan:
   - Program menjadi lebih pendek karena satu baris instruksi bisa melakukan banyak hal sekaligus.
   - fleksibel, karena tidak menimpa data sumber (data A dan B tetap aman).

   Kelemahan:
   - Ukuran setiap instruksi menjadi sangat panjang (membutuhkan banyak bit untuk menampung 3 alamat).
   - Unit pemrosesan di CPU menjadi lebih kompleks.

2. **Format Instruksi 2 Alamat**
   Format yang paling umum digunakan dalam prosesor modern. Salah satu alamat berfungsi ganda sebagai sumber sekaligus tempat menyimpan hasil.
   Contoh: *ADD A, B*
   (Menambahkan A dengan B, lalu menyimpan hasilnya kembali ke A)
   Kelebihan:
   - Ukuran instruksi lebih hemat bit dibandingkan format 3 alamat
   - keseimbangan yang baik antara panjang program dan kecepatan prosesor

   Kelemahan:
   - Salah satu data sumber asli akan hilang karena ketimpa oleh hasil perhitungan.
   - Terkadang membutuhkan instruksi tambahan seperti *MOV* untuk mengamankan data sebelum diproses.
3. **Format Instruksi 1 Alamat**
   Format ini menggunakan register khusus yang disebut Accumulator (AC) secara implisit. CPU mengasumsikan bahwa salah satu data sudah ada di dalam Accumulator.
   Contoh: *ADD B*
   (Tambahkan isi Accumulator dengan B, lalu simpan hasilnya kembali ke Accumulator).
   Kelebihan:
   - Insturksi sangat ringkas dan sederhana hanya perlu mencantumkan satu alamat memori.
   - Sirkuit kontrol CPU menjadi lebih sederhana.

Kelemahan:

- Program menjadi sangat panjang karena setiap langkah kecil membutuhkan instruksi terpisah (harus mengisi Accumulator dahulu, baru diproses).
- Accumulator sering menjadi bottleneck karena semua data harus lewat sana.

4. **Format Instruksi 0 Alamat**

Format tidak mencantumkan alamat memori sama sekali dalam instruksi aritmatikanya. Format ini menggunakan struktur data **Stack.** Alamat dianggap sudah diketahui secara implisit, yaitu dua data teratas di tumpukkan.

Contoh: *ADD*

(Ambil dua angka teratas dari stack, jumlah, dan masukkan kembali ke puncak stack).

Kelebihan:

- Instruksi sangat pendek (hanya butuh beberapa bit untuk kode operasi).
- Sangat efisien untuk mengevaluasi ekspresi matematika kompleks.

Kelemahan:

- Manajemen stack di dalam memori bisa menjadi rumit.
- Eksekusi bisa lebih lambat jika data harus terus menerus push/pop dari memori utama ke stack.